

DRONE GCS // DETECTION CONSOLE

Interceptor Drone Project

A real-time drone detection ground control console — from Roboflow dataset to a live, deployable web product.

Phase	1 — Detection (complete)
Next phase	2 — Mission Planner arm / disarm
Stack	Flask · OpenCV · Roboflow Inference API · Chart.js
Repository	github.com/ayodhya11/interceptor-drone-project

```
Directory: C:\Users\AYODHYA\Downloads

Mode                LastWriteTime         Length Name
----                -
-a-----         05-07-2026    20:49         18688 drone-detection.zip

PS C:\Users\AYODHYA\downloads\drone-detection_2\drone-detection> dir "C:\Users\AYODHYA\Downloads\drone-detection3.zip"
dir : Cannot find path 'C:\Users\AYODHYA\Downloads\drone-detection3.zip' because it does not exist.
At line:1 char:1
+ dir "C:\Users\AYODHYA\Downloads\drone-detection3.zip"
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\AYODHY...-detection3.zip:String) [Get-ChildItem], ItemNotFound
Exception
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.GetChildItemCommand

PS C:\Users\AYODHYA\downloads\drone-detection_2\drone-detection> cd C:\Users\AYODHYA\Downloads
>> findstr /S /M /C:"Upload images" *.html
drone-detection_2\drone-detection\drone-detection\templates\dashboard.html
FINDSTR: Cannot open flytbase-20260627T155446Z-3-001\flytbase\Overview  FlytBase Customer Health.html
PS C:\Users\AYODHYA\Downloads> findstr /S /M /C:"Upload images" *.html
drone-detection_2\drone-detection\drone-detection\templates\dashboard.html
FINDSTR: Cannot open flytbase-20260627T155446Z-3-001\flytbase\Overview  FlytBase Customer Health.html
PS C:\Users\AYODHYA\Downloads> |
```

The dashboard — live model status, project aim, and detection history in one console.

What this project is

This project detects drones in real time — from a live webcam feed or an uploaded image/video — using a computer vision model hosted on Roboflow. It's built as the foundation for a larger goal: on the first confirmed detection, automatically connect to **Mission Planner** and arm/disarm the target drone. That integration is Phase 2; everything in this document is the completed Phase 1.

Why it's more than a notebook

Most drone-detection projects stop at a Jupyter notebook that draws boxes on a test image. This one goes further: it's a real product someone could actually use — a login, a dashboard, live detection from any visitor's own camera, automatic result logging, and a one-click data export. The goal was to practice the full path from a trained model to something deployable, not just the modeling step.

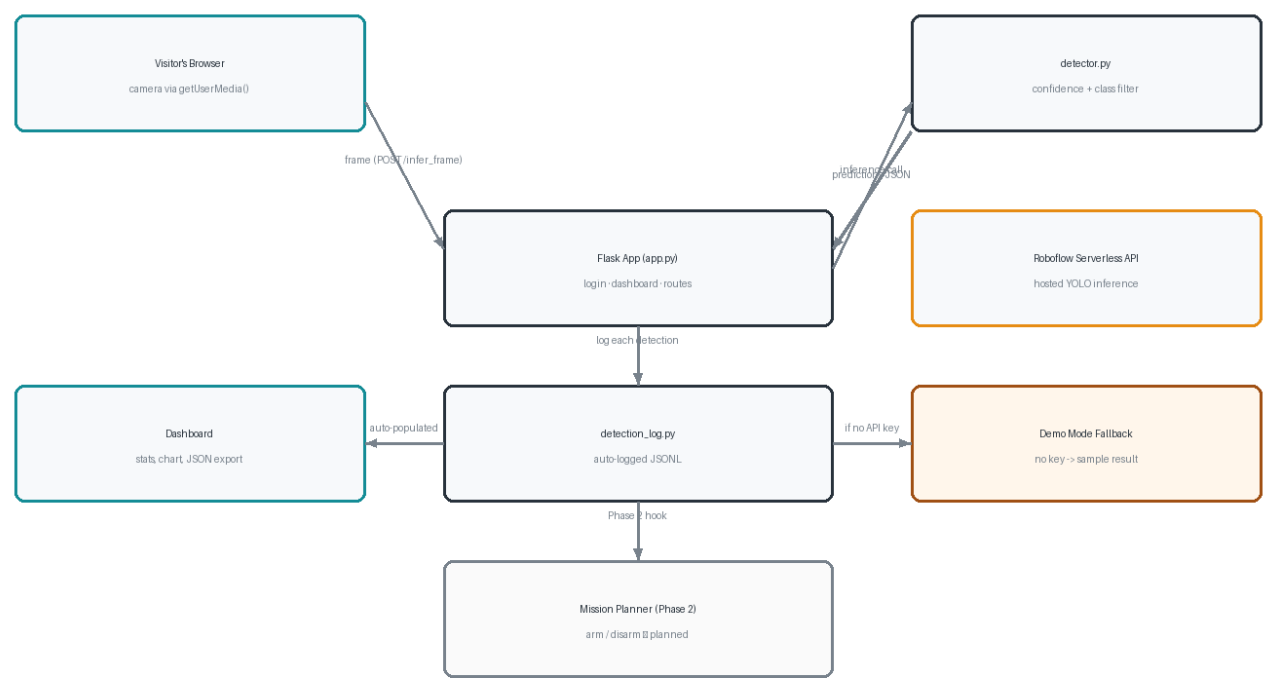
Pipeline

Step	Tool
1. Collect & label images	Roboflow
2. Host the trained model	Roboflow serverless inference API
3. Build the product	Flask + browser-based camera capture
4. Log & surface results	Auto JSONL log → dashboard chart & table
5. Deploy publicly	Render (Git-based, HTTPS out of the box)

Design goals

- **Resilient by default** — the app never looks broken. If no API key is configured, a clearly labeled demo mode shows a sample detection instead of an error page.
- **Correct under real usage** — detection state (confirmations, alerts) is scoped per visitor session, not a single shared global, so multiple people can use the live deployment at once without interfering with each other.
- **Honest about limitations** — false positives from a public/community-trained model are a dataset limitation, not something UI code can fully mask; the app is upfront about that rather than hiding it.

How it fits together



Request flow: a visitor's browser captures a frame from their own camera, sends it to Flask, which calls Roboflow's hosted model and logs the result.

Key engineering decisions

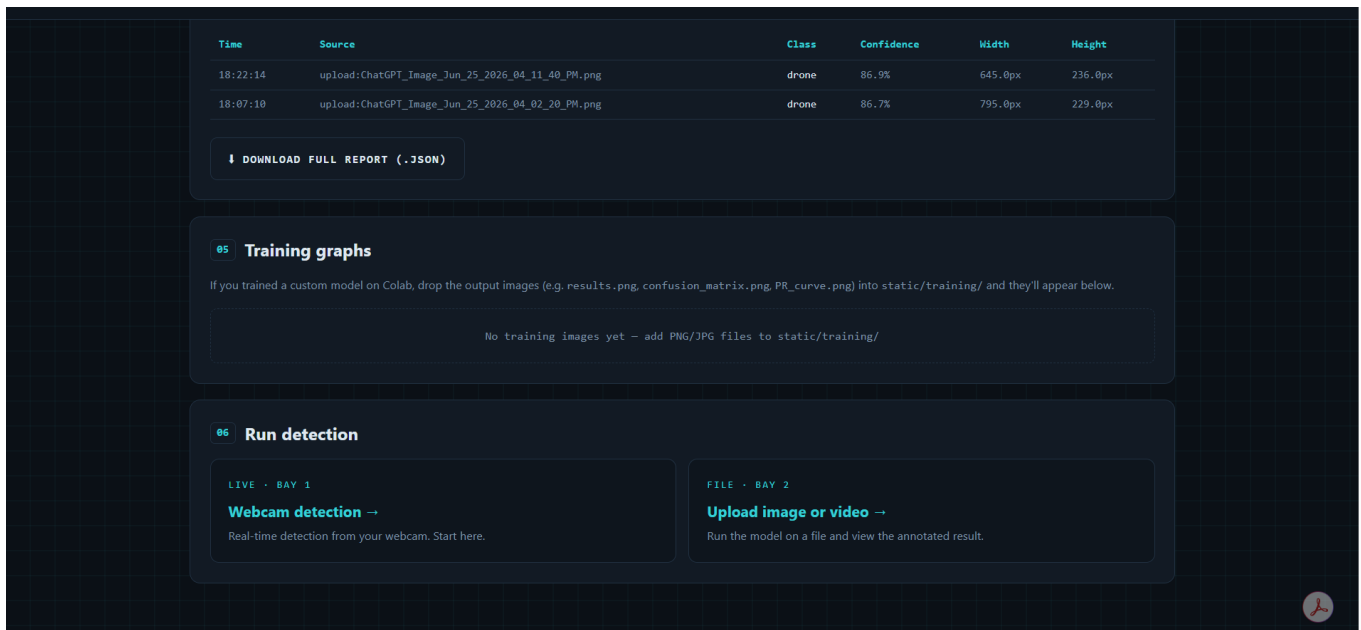
Decision	Why
Browser captures the camera, not the server	This server has no physical webcam once deployed to the cloud. Using getUserMedia() means live video.
Two-loop capture design	A fast client-side render loop (~60fps) keeps video smooth; a slower loop (~1.4/sec) calls the server.
Per-session state, not a global	Detection confirmation state lives in the Flask session (cookie-based), so concurrent visitors on a page don't interfere.
Confidence threshold + class filter	Configurable via .env; raises the bar before a detection counts, reducing false positives from a generic model.
Temporal confirmation (2-in-a-row)	A detection must appear in two consecutive frames before the alert / Mission Planner hook fires — reduces noise.
Demo mode fallback	No API key? The app substitutes a clearly labeled sample result instead of erroring — the product still works.

03 · Feature walkthrough

What it actually does

Detection results — logged automatically

Every detection, from the webcam or an uploaded file, is logged with its class, confidence, and bounding-box size — with zero manual entry. The dashboard turns this into a live chart and a recent-detections table, and a single button exports the full history as JSON.



The Detection Results panel — auto-populated from real inference calls.

Verified detection pipeline

Below: a real test run against the hosted model, returning a genuine prediction (class, confidence, and box dimensions) — confirming the inference pipeline works end-to-end before it was wired into the web app.

```
PS C:\Users\AYODHYA\downloads\drone-detection\drone-detection> python test_inference.py drone.png
Model: drone-uskpc/1
Running inference on: drone.png

[
  {
    "x": 398.5,
    "y": 204.5,
    "width": 525.0,
    "height": 243.0,
    "confidence": 0.8126635551452637,
    "class": "drone",
    "class_id": 0,
    "detection_id": "a7679cf8-b053-4dc2-b16b-f18b8cf8b5d1"
  }
]

Detections: 1
Annotated image saved to: drone_annotated.jpg
PS C:\Users\AYODHYA\downloads\drone-detection\drone-detection> |
```

A real detection: class "drone", confidence 0.81.

What's done, what's next

Phase	Status	Description
1 — Detection	Complete	Login, dashboard, webcam + upload detection, auto-logged results, demo mode, public
2 — Mission Planner	Planned	On first confirmed detection, connect via pymavlink and arm / disarm the target drone
3 — Tracking (future)	Idea	Extend from single-frame detection to multi-frame tracking of a confirmed target

What I'd build next

- Wire the Mission Planner hook (already stubbed in code) to pymavlink for real arm/disarm.
- Add authentication beyond email-only (e.g. magic links) if this ever handles real flight hardware.
- Move detection logs from a flat file to a small database once traffic outgrows a single-file log.

Links

Repository: github.com/ayodhya11/interceptor-drone-project

Live demo: *add your Render URL here once deployed*

Built as Phase 1 of an interceptor-drone system — dataset, hosted inference, and a real product wrapped around it.